

Applied Agentic AI for PMs/TPMs

Capstone Project Playbook

A Step-by-Step Guide to Completing Your Project

Common guide for CalendarMate, SalesGenie, PRD Genie, and Mira

Use this document if you need structured guidance on how to approach your capstone. Every step has guiding questions. Answer them, and you will have your solution.

How to Use This Guide

This guide is optional. Your capstone problem statement document has everything you need to complete the project. This guide is for learners who want a structured path with guiding questions at each step.

The guide follows 7 steps, from understanding your problem to submitting your work. At each step, you will find questions in blue boxes. Answer those questions and you will naturally arrive at your design decisions, implementation choices, and evaluation strategy.

The questions are the same regardless of which project you picked. Project-specific examples are provided in orange boxes to show how the question applies to CalendarMate, SalesGenie, PRD Genie, or Mira.

Step	What You Will Decide
Step 1: Understand the Problem	What are the core pain points? What does success look like?
Step 2: Select Your Tools	Which platform, LLM, APIs, and observability tool?
Step 3: Design Your Agent System	How many agents? Which orchestration pattern? What does each agent do?
Step 4: Build Core Capabilities	Start simple, get one flow working, then expand
Step 5: Run the Baseline Test Dataset	Test all inputs, document outputs, check expected keywords
Step 6: Evaluate and Iterate	Connect observability, analyze traces, fix prompts
Step 7: Document and Submit	Architecture writeup, cost analysis, reflections

Step 1: Understand the Problem

Before you build anything, make sure you understand what you are solving and for whom. Read your problem statement carefully and answer these questions.

Guiding Questions

- ? Who is the user of this system? What is their role and daily workflow?
- ? What are the top 3 tasks that waste the most time in their current process?
- ? What does the user currently do manually that your AI assistant will automate?
- ? What does a successful output look like for each core capability? Can you write one example by hand?
- ? What are the risks if the AI produces wrong output? (e.g., scheduling a meeting at the wrong time, recommending an out-of-stock product, inventing a requirement that was never discussed)

CalendarMate example:

User is a PM/TPM spending 2 hours/day on meeting logistics. Top 3 wastes: morning triage (30-45 min), manual scheduling (back-and-forth emails), no follow-up tracking. Risk: AI invents a meeting that does not exist or double-books a slot.

SalesGenie example:

User is a sales rep reading 50+ inquiry emails/day. Top 3 wastes: manually reading emails (2-3 hours), inconsistent lead scoring, manual product matching. Risk: AI recommends a product not in the catalog or misclassifies a hot lead as cold.

PRD Genie example:

User is a PM/TPM translating meeting discussions into PRDs. Top 3 wastes: rewriting transcripts into structured docs, inconsistent PRD formats, lost requirements. Risk: AI invents a requirement that was never stated in the transcript.

Mira example:

User is a PM/TPM managing 3-5 projects simultaneously. Top 3 wastes: creating project plans from scratch (3-4 hours each), compiling weekly status reports from task boards (1-2 hours), inconsistent risk assessments. Risk: AI fabricates milestones not in the project description or invents task statuses not in the task board data.

Checkpoint:

Write a 3-sentence summary of the problem you are solving, who you are solving it for, and what the biggest risk is. This becomes the opening of your writeup.

Step 2: Select Your Tools

You need to choose tools for 5 categories. For each, answer the guiding questions below. Document your choices and reasoning — this goes into your architecture writeup.

2.1: Workflow Platform

- ? Which platform have I used in class? Am I comfortable building agents in it?
- ? Does it support the APIs I need? (Calendar, Email, File ingestion)
- ? Can it connect to an observability tool? (LangWatch, Langfuse)

Platform	Best For
n8n	Visual workflow builder. Native Google Calendar and Gmail nodes. Good for CalendarMate and SalesGenie.
LangFlow	Visual agent builder. Good for multi-agent pipelines with custom prompts. Good for PRD Genie.
Custom code (Python)	Maximum flexibility. Requires coding. Good if you want full control.

2.2: LLM Model

- ? Do I need a cheap model for high-volume tasks (classifying 50 emails/day) or a smart model for complex reasoning (generating a PRD)?
- ? What is my budget? How many API calls will my system make per day?
- ? Should different agents use different models? (e.g., cheap model for classification, smarter model for generation)

Model	When to Use
gpt-4o-mini / gpt-4.1-mini	Cheap, fast. Good for classification, extraction, simple generation. Best default choice.
gpt-4o / gpt-4.1	Smarter, more expensive. Use for complex reasoning, evaluation judging, or when output quality is critical.
Claude (Anthropic)	Strong at long documents and careful reasoning. Good for PRD generation.
Gemini (Google)	Good integration with Google Workspace. Consider for CalendarMate if using Google APIs.

2.3: APIs and Integrations

- ? What external services does my agent need to read from? (Calendar, Email, Files, CRM)
- ? What external services does my agent need to write to? (Create events, send emails, update spreadsheets)
- ? Do I need OAuth2 authentication? Which provider? (Google, Microsoft)

CalendarMate:

Read: Google Calendar (events, availability), Gmail (unread emails). Write: Google Calendar (create events), Gmail (send follow-ups). Auth: Google OAuth2.

SalesGenie:

Read: Email inbox or file upload (inquiry emails), CSV (product catalog). Write: Email (responses), Sheets/CSV (CRM export). Auth: depends on email provider.

PRD Genie:

Read: File upload (transcripts, notes, briefs). Write: Markdown or text (PRD output). Auth: usually not needed if using file upload.

Mira:

Read: Text files (project descriptions), CSV files (timelines, risks, task boards). Write: Markdown or text (plans, reports, risk matrices). Auth: Trello API key if using live board, otherwise not needed for CSV upload.

2.4: Observability Tool

- ? Which tracing tool have I used in class? (LangWatch, Langfuse)
- ? Can I connect it to my chosen workflow platform?
- ? What do I need to see in traces? (input/output per agent, token usage, latency, cost)

Quick decision:

Use whichever tool you set up during the evaluation session in class. Both LangWatch and Langfuse work. The connection method depends on your platform (environment variables for LangFlow, built-in integration for n8n).

2.5: Document Your Choices

Before moving to design, write down your tool selections in a table like this:

Category	Your Choice + Why
Workflow Platform	e.g., n8n — I used it in class, it has native Calendar nodes
LLM	e.g., gpt-4o-mini — cheap, fast enough for classification and generation
APIs	e.g., Google Calendar + Gmail via OAuth2
Observability	e.g., Langfuse — set up in class, 3 env vars in LangFlow

This table goes directly into your writeup.

The rubric awards points for tool selection rationale. “I chose X because...” is more valuable than just listing the tool.

Step 3: Design Your Agent System

This is the most important design step. You are deciding how many agents, what each one does, and how they work together. The orchestration pattern you select must be documented and justified.

3.1: Break the Problem into Tasks

?

What are the distinct tasks my system needs to perform? List them.

?

Does each task need its own agent, or can some be combined into one?

?

What is the input and output of each task?

CalendarMate tasks:

(1) Classify user intent (briefing vs scheduling vs email). (2) Pull calendar events. (3) Check availability. (4) Create events. (5) Read emails. (6) Summarize emails. (7) Generate follow-ups.

SalesGenie tasks:

(1) Parse inquiry email. (2) Extract lead fields (name, budget, product interest). (3) Classify lead tier. (4) Match to catalog. (5) Draft response. (6) Aggregate weekly stats.

PRD Genie tasks:

(1) Parse transcript/notes. (2) Extract requirements (stated vs ambiguous). (3) Generate structured PRD. (4) Break into user stories. (5) Identify gaps.

Mira tasks:

(1) Classify user request type (plan vs risk vs status vs milestone). (2) Parse project description and timeline. (3) Generate structured project plan. (4) Analyze risks from project data. (5) Read task board CSV. (6) Generate weekly status report. (7) Flag at-risk milestones.

3.2: Choose Your Orchestration Pattern

Your problem statement lists three patterns. Here is how to decide:

?

Does every input follow the exact same sequence of steps? If yes → Sequential Pipeline.

?

Can inputs be very different types that need different handling? If yes → Router / Dispatcher.

?

Does a single request require breaking it into subtasks handled by multiple agents? If yes → Hierarchical.

Pattern	Choose When...	Project Fit
Sequential Pipeline	Every input goes through the same stages in order	SalesGenie (every email: parse → qualify → recommend), PRD Genie (every transcript: extract → PRD → stories)
Router / Dispatcher	Inputs are varied and need different specialist agents	CalendarMate (schedule vs summarize vs briefing), Mira (plan vs risk vs status)
Hierarchical	One complex request needs multiple agents working together	CalendarMate ('prepare me for tomorrow' → briefing + email + conflicts)

Common mistake:

Do not pick a pattern just because it sounds sophisticated. A simple sequential pipeline that works reliably is better than a complex hierarchical system that breaks. Justify your choice based on your project’s actual input patterns.

3.3: Define Each Agent

For every agent in your system, answer:

- ? What is this agent's single responsibility? (one job per agent)
- ? What input does it receive? From which agent or trigger?
- ? What output does it produce? In what format?
- ? Does it need tools? (Calendar API, Email API, file reader, web search)
- ? What should it do when the input is ambiguous or insufficient? (This is the hallucination guardrail.)

The last question is critical.

Most hallucination happens because the agent was never told what to do when it does not have enough information. Your prompt must include a rule like: 'If the input is too vague to determine X, say UNKNOWN. Do NOT invent X.'

3.4: Draw Your Architecture Diagram

Create a visual diagram showing:

- Every agent as a box with its name and role
- Arrows showing data flow between agents
- External APIs shown as separate boxes connected to the agents that use them
- The trigger (chat input, schedule, email trigger) at the top
- The output (user response, email, file) at the bottom

You can use any tool: Excalidraw (free, in-browser), draw.io, Miro, PowerPoint, or even paper and phone camera. The diagram goes into your writeup and submission.

Step 4: Build Core Capabilities

Start building. The key principle: get one flow working end-to-end before building everything.

4.1: Start with the Simplest Core Capability

- ? Which core capability is the simplest to build and test?
- ? Can I get it working with a single agent first, then add more?
- ? What is the minimum test I can do to verify it works? (one input, one expected output)

CalendarMate:

Start with Daily Briefing. It only needs to read calendar events and format them. One agent, one API call, one output.

SalesGenie:

Start with Lead Qualification on a single sample email. One agent, one input, check if it extracts the right fields.

PRD Genie:

Start with Requirement Extraction on one transcript. One agent, one input, check if it separates stated from ambiguous.

Mira:

Start with Project Plan Generation using the ABCDE Ltd. project description file. One agent, one text file input, check if it produces phases and milestones grounded in the description.

4.2: Write Your System Prompts

Each agent needs a system prompt. Here is how to write a good one:

- Start with **ROLE**: one sentence describing what this agent does
- Define **INPUT**: what the agent receives
- Define **OUTPUT**: the exact format you expect (list every field)
- Add **RULES**: what the agent must NOT do (the hallucination guardrails)
- Add the **VAGUE INPUT RULE**: what to do when input is insufficient

Prompt template:

ROLE: You are a [job title]. **INPUT:** [what you receive]. **OUTPUT:** Return EXACTLY these fields: [list fields]. **RULES:** 1) Do NOT invent [X]. 2) If [condition], say UNKNOWN. 3) Only use information explicitly stated in the input.

4.3: Add Capabilities One at a Time

After the first capability works:

- Add the second core capability. Test it independently.
- Add the third core capability. Test it independently.
- Connect them through your orchestration pattern.
- Test the full pipeline end-to-end with at least 3 inputs.
- Then add one extended capability.

Do not build everything at once.

The most common failure is trying to build all agents simultaneously, then debugging a broken pipeline with no idea which agent is failing. Build and test incrementally.

Step 5: Run the Baseline Test Dataset

Your problem statement includes a baseline test dataset with 10-12 test inputs and expected output criteria. This is how TAs will evaluate your work.

- ? Have I run every single test input from the baseline dataset through my pipeline?
- ? For each output, does it contain the expected keywords/elements listed in the 'Expected Output Must Contain' column?
- ? For vague/edge-case inputs, does my system correctly ask for clarification instead of inventing details?
- ? Have I documented each input and its corresponding output? (screenshots or copy-paste into a document)

How to Document Test Results

Create a simple table for each test input:

Field	Value
Test ID	T1
Input	(the test input text)
Output	(full pipeline output, copy-pasted)
Expected keywords present?	Yes: [list which ones]. Missing: [list if any].
Hallucination detected?	No / Yes: (describe what was invented)
Pass/Fail	Pass

This documentation is a submission requirement.

The rubric allocates 5 points specifically for 'baseline dataset tested with documented results.' TAs will check each test input against the expected output criteria.

Step 6: Evaluate and Iterate

Connect your system to an observability tool and use the traces to find and fix problems.

6.1: Connect Observability

- ? Is my tracing tool connected? Do traces appear for every pipeline run?
- ? Can I see the input and output of each individual agent in the trace?
- ? Can I see token usage, cost, and latency per agent?

6.2: Analyze Your Traces

- ? Look at a trace for a vague/edge-case input. Did any agent invent information?
- ? Look at a trace for a detailed input. Are the outputs grounded in the actual data?
- ? Compare cost between good inputs and vague inputs. Is the vague input costing more tokens? (Hallucination often generates longer, more expensive outputs.)
- ? What is the total cost per pipeline run? Multiply by expected daily volume to get cost per user per day.

6.3: Fix and Re-Test

If you found hallucination or incorrect outputs:

- Identify which agent produced the wrong output (check each agent's output in the trace)
- Modify that agent's system prompt to add a guardrail (e.g., 'If budget is not mentioned, say UNKNOWN')
- Re-run the same test input and compare before vs after in your traces
- Document the change: what you modified, why, and how the output improved

This is the evaluation-optimization loop from class:

Evaluate → Find failure → Fix prompt → Re-evaluate → Confirm improvement. If you do this even once and document it, you are demonstrating the core skill of the course.

6.4: Define Production Metrics

Your writeup must include at least 3 metrics you would track in production. Answer:

- ? How would I measure whether the system is accurate? (e.g., classification accuracy, scheduling success rate, requirement extraction completeness)
- ? How would I measure whether the system is trustworthy? (e.g., hallucination rate, false-positive rate)
- ? How would I measure whether users find it useful? (e.g., time saved per day, adoption rate, user satisfaction score)

CalendarMate metrics:

(1) Scheduling accuracy: % of events created at the correct time with correct attendees. (2) Hallucination rate: % of briefings that mention non-existent meetings. (3) Time saved: minutes saved per user per day vs manual workflow.

SalesGenie metrics:

(1) Classification accuracy: % of leads correctly classified as Hot/Warm/Cold. (2) Recommendation relevance: % of recommendations that match the catalog and budget. (3) Response time: seconds from email received to recommendation generated.

PRD Genie metrics:

(1) Extraction completeness: % of stated requirements correctly captured. (2) Hallucination rate: % of PRD items not traceable to source transcript. (3) Format compliance: % of PRDs that follow the template structure.

Mira metrics:

(1) Plan groundedness: % of milestones in generated plan that are traceable to the project description. (2) Risk relevance: % of identified risks that are specific to the project vs generic. (3) Status accuracy: % of task counts in status reports that match the actual task board data.

Step 7: Document and Submit

Gather all your work into a clean submission package.

7.1: Architecture Writeup (1-2 pages)

Your writeup should cover:

- Problem summary (3 sentences from Step 1)
- Architecture diagram (from Step 3)
- Tool selection rationale with the table from Step 2
- Orchestration pattern choice and justification
- Agent descriptions: what each agent does, its input/output, its key prompt rules
- Cost analysis: estimated cost per user per day (tokens × price, times daily volume)
- Evaluation strategy: your 3 production metrics from Step 6
- Testing observations: what worked, where the system hallucinated, what you fixed

7.2: Submission Checklist

- Workflow exported as JSON, uploaded to GitHub repository with a README
- Architecture diagram included (any format)
- Baseline test dataset results documented (all inputs with outputs)
- Screenshots: workflow canvas, system in action, observability traces
- Architecture writeup (1-2 pages) with all sections listed above
- All assignment questions answered (Q1, Q2, Q3, Q4 as per your project)
- Slide deck summarizing problem, approach, and outcomes
- GitHub repository public link submitted

7.3: Cost Analysis Template

Use this template for your cost estimate:

Cost Item	Estimate
LLM tokens per pipeline run	e.g., ~2,000 input + ~1,500 output = 3,500 tokens
Cost per run at gpt-4o-mini pricing	e.g., \$0.0005 per run
Estimated runs per user per day	e.g., 15 runs (briefings + scheduling + emails)
Cost per user per day	e.g., \$0.0075/day = ~\$0.15/month
API calls (Calendar, Email)	Free within Google API quotas for typical usage
Observability tool	Free tier sufficient for evaluation
Total estimated monthly cost per user	e.g., ~\$0.20/month

Optional: Fine-Tuning

Fine-tuning is not required but is recognized in scoring. If you want to attempt it:

- ? Which agent in my system hallucinates the most? That is the best fine-tuning candidate.
- ? Can I create 10-15 training examples showing the correct behavior? (input + desired output pairs)
- ? For vague inputs specifically, can I create 5 examples showing the correct 'insufficient data' response?
- ? How will I measure improvement? (Run the same test dataset before and after fine-tuning.)

If you completed the fine-tuning session in class, you already have the scripts and process. Apply the same approach to your capstone agent: create JSONL training data, run the fine-tuning script, swap the model ID in your workflow, and re-evaluate.

Using AI to help:

You can use Claude or ChatGPT to help you generate training data. Prompt: 'I need 15 JSONL training examples for an agent that classifies customer inquiry emails as Hot/Warm/Cold. Include 10 detailed examples and 5 vague examples where the agent should output UNCLEAR. Format each as a system/user/assistant message triple.'

Common Mistakes to Avoid

- 1. Building everything at once:** Start with one agent, one capability, one test. Get it working, then expand. Do not build 5 agents and debug them all simultaneously.
- 2. No hallucination guardrails:** If your prompt does not say what to do when the input is vague, the LLM will invent details. Every agent prompt needs a rule for insufficient input.
- 3. Not running the baseline test dataset:** TAs will check your outputs against the expected keywords. If you skip this, you lose 5 points automatically.
- 4. Choosing a pattern without justification:** 'I chose the router pattern' is not enough. 'I chose the router pattern because user requests in CalendarMate can be very different types (briefing, scheduling, email) and each needs a different agent' is what the rubric rewards.
- 5. No cost analysis:** The rubric specifically asks for cost per user per day. Count your tokens, multiply by pricing, multiply by expected daily volume. It takes 5 minutes.
- 6. Skipping observability:** Without traces, you cannot prove your system works. Connect to LangWatch or Langfuse early, not the night before submission.
- 7. Not documenting what went wrong:** The writeup rewards honesty about failures. 'The scheduler agent invented a meeting time when the calendar was empty' + 'I fixed it by adding this rule to the prompt' is more valuable than pretending everything worked perfectly.